Intro.

- Able to fit on all devices [Responsive web-design]

Strategies

1. Responsive web design → using CSS Rules
   (eg) grid, flexbox, Media rules.

   Now usually used this.
   but Amazon still using Server Sniffing.

2. Server Sniffing (Dynamic Serving) → $\boxed{\text{servers}}$

   well check which device then diff HTML, CSS, JS.

3. Separate URLs for mobile and Desktop.

   m.facebook.com.

   only url is diff.

Responsive Layout Techniques

- Grid - Use fr
- Flex - using flex wraps

} These are fluid designs.

- Relative Units → %, vw, vh, rm, rm

Viewport

- making size of content – proportional to screen size. → creating relative viewport.

very simple to include this.

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
</head>
```

initial-scale = 1.0 → 100% screen utilized

if 2.0 → 200% (zoom factor)

Media Queries

- written as @media

→ mediatype { screen, print, All }

→ mediafeature {resolution of O/P device etc}

→ width, height, aspect ratio, orientation, resolution.

should always be included in parantheses.

@media screen and (max-width : 200 px)

Ex

device's max width → 780 px or less → If size is less than 780 px, I want the rule to apply.

```
@ media screen and (max-width : 780px) {
    # div1 {
        background-color : aqua;
        }
    # text {
        column-count : 2;
        }
}
```

```
@ media screen and (max-width : 780 px) and (min-width : 600 px) {
```

will create range from 680 px - 780 px

Note: overlapping ranges should be avoided.

```
@ media screen and (width <= 780px)
                   (680px <= width <= 780 px)
```

(For giving range)

Now all browsers support this

Q Which → media query will apply the styles only to screen devices with a width of 800 px or more.

↓

@ media only screen and (min-width: 800 px)

↓

ensures that media query applies only to screen devices not print or other media types.

Mobile First Approach Vs Desktop first Approach

Mobile first Approach → Progressing from smaller to bigger screens.

@ media screen and (min-width : 480 px) {

}

→ Decide these break points very carefully

(Con search)

- Note: → default font size of html is 16px.

↳ If I want it to look same everywhere → then →

```
html {
        font-size : 2r ;
    }
```

$\boxed{1\,vw}$ → 1% of viewport

vw = 100px

vh = 200px

- min-width, max-width → can be used with img etc as well.

Some examples

```
• image {
    width : 50% ;
    height : auto ;
  }
```

→ Percentages for image width might not scale proportionally. Consider max-width or viewport units for better control.

```
h1 {
font-size : 2em ;
  }
```

→ em inherits base font size which might cause inconsistencies. Use rem for relative sizing based on root font size.

- creating responsive rectangle < height – 30% of smaller viewport dimension. → 30 vmin
  width – 50% of larger viewport dimension. → 50 vmin

Media Queries features

```
@ media screen and ( orientation : landscape) {
                     (width > height)
                     portrait
    # div1 {
        background-color : black ;
    }
```

or

```
@ media screen and (min-aspect-ratio : 1/1)
```

```
@ media (orientation : landscape) and (min-width : 480px)
```

↓

specifies both orientation and min^m width, ensuring styles are applied only to devices that meet both criteria.

```
@ media screen and (orientation : landscape) and (max-aspect-ratio: 16/9) {
    • header {
        height : 50px;
    }
}
```

→ when header height be set to 50px

→ This will apply styles only when both conditions are met

Container Queries

It is a way to apply styling to the element based on element's container.

Note : h3 {
font-size : clamp (1.4 rem, 4vw, 2.5rem) ;
}

min actual value max

- article -container {
  container-type : inline-size;
  container- name : article;
  }

container : article / inline-size

@ container article (inline- size 750px) {
article {
}
}

Same Rules
grid- template- columns : 500 px;

@ container article (inline- size 800 px) {
article {
grid ——
}
}

↳ Use container-units (eg) for viewport → vw etc

↳ for container: cqw → container-width

cqh → " - height

cqi → 1% of container's inline size

cqb → " " " block "

cqmin → smaller value of cqw or cqh.

cqmax → larger " " " " ".

```
@container wrapper-container (inline-size: 600px) and (block-size: 400px) {
    .item {
        font-size: 1.5rem;
    }
}
```

It sets the container to measure both its inline size (width) and block size (height) for container queries.